Econ 390 SP26 Practice Final Exam

A. [2 Points Each] Assume no other code was run.

```
codes = {"CPIAUCSL":"CPI", "CUSR0000SACL1E":"Core CPI",
"CORESTICKM159SFRBATL":"Sticky Price CPI"}
fred_CPI = web.DataReader(list(codes.keys())[:2], 'fred',
start="2025-01-01", end="2026-03-01")
fred_CPI = fred_CPI.rename(columns = codes).pct_change(periods
= 1)*100
```

1. What website is the API for?

   _____

2. How many variables are included in the query?

   _____

3. What level of frequency is the data?

   _____

4. How many observations per variable are returned from the query?

   _____

5. What are the dimensions of non-N/A elements in the dataset after all the code is run (assume data for all periods are available)?

   _____

6. The code will run with no errors. True/False

B. [2 Points Each] True or False. If False, fix the statement to make it True.

7. The Lasso can also be useful for "feature selection". True/False

_____

8. When running the Lasso, the Mean Squared Error for the test set will almost always be weakly less than the training set. True/False

_____

9. Categorical variables work with `sklearn.linear_model.Lasso` True/False

_____

10. Heatmap plots are specifically available via Seaborn. True/False

_____

11. Stylesheets as we learned them work for any plotting package in Python. True/False

_____

12. `type(seaborn.jointplot(.).ax_joint)` is a matplotlib axis. True/False

_____

13. The Difference in Differences estimate is causal if the assumptions are met. True/False

_____

14. Regular Expression is Python specific. True/False

_____

C. [1 Point Each] Order the lines of code by line number (1-10) to produce the following results:

```
                                Income              Weeks Worked
                                 count          mean        count       mean
Sex    Race
Female Black                       978  34392.551125         1166  27.362779
       Hispanic                    742  33336.653639          924  25.594156
       Mixed Race                   30  37669.000000           43  23.465116
       Non-Black / Non-Hispanic   1675  45859.256119         2252  25.115453
Male   Black                       885  34346.823729         1169  23.650128
       Hispanic                    677  51135.169867          977  25.309110
       Mixed Race                   26  55846.115385           40  24.675000
       Non-Black / Non-Hispanic   1700  73112.743529         2413  26.469540
Sex     Race
Female  Black                       978.0
        Hispanic                    742.0
        Mixed Race                   30.0
        Non-Black / Non-Hispanic    1675.0
Male    Black                       885.0
        Hispanic                    677.0
        Mixed Race                   26.0
        Non-Black / Non-Hispanic    1700.0
Name: (Income, count), dtype: float64
Sex     Race
Female  Black                      27.362779
        Hispanic                   25.594156
        Mixed Race                 23.465116
        Non-Black / Non-Hispanic   25.115453
Male    Black                      23.650128
        Hispanic                   25.309110
        Mixed Race                 24.675000
        Non-Black / Non-Hispanic   26.469540
Name: (Weeks Worked, mean), dtype: float64
```

15. `race_codes = {1:"Black", 2:"Hispanic", 3:"Mixed Race", 4:"Non-Black / Non-Hispanic"} ___`
16. `nlsy["Sex"] = nlsy["Sex"].replace(sex_codes) ___`
17. `print(nlsy_grouped.describe().iloc[:,0]) ___`
18. `nlsy = pd.read_csv("https://m-mcmain.github.io/files/Econ390SP26/nlsy_wage_educSex.csv", na_values=-5, usecols = variables.keys()).rename(columns = variables) ___`
19. `variables = {"R0536300":"Sex", "U5753500":"Income", "Z9061912":"Weeks Worked", "R1482600":"Race"} ___`
20. `print(nlsy_grouped.describe().iloc[:,9]) ___`
21. `sex_codes = {1:"Male", 2:"Female", 0:"No Information"} ___`
22. `print(nlsy_grouped.agg(["count", "mean"])) ___`
23. `nlsy_grouped = nlsy.groupby(["Sex", "Race"]) ___`
24. `nlsy["Race"] = nlsy["Race"].replace(race_codes) ___`

Write the results of running the following code (assuming 15-24 was run in the correct order) or “Error” if it would cause an error.

25. `nlsy_grouped.agg(["count", "mean"]).iloc[0,0]`

    _____

26. `nlsy_grouped.agg(["count", "mean"])[8,2]`

    _____

27. `nlsy_grouped.describe().iloc[:,0][0]`

    _____

D. [2 Points Each] Write down the results of running the code. Assume the following code has already been run.

```
String1 = "In 2025, 5.7 percent of families included an unemployed
person, up from 5.3 percent in 2024, the U.S. Bureau of Labor
Statistics reported today."
String2 = "Total nonfarm payroll employment increased by 178,000
in March, and the unemployment rate changed little at 4.3 percent,
the U.S. Bureau of Labor Statistics reported today. Job gains
occurred in health care, in construction, and in transportation
and warehousing."
```

28. `print(re.findall("\d\.\d",String1))`

    _____

29. `print(re.findall("\d+[,\.]+\d+",String2))`

    _____

30. `print(re.findall("[A-Z][a-z]+",String2))`

    _____

31. `print(re.findall("(?:^|\.\s)([A-Z][a-z]+)",String2))`

    _____

32. `print(re.findall("(\d+\.*\d*)(?:%|\spercent)",String1))`

    _____

33. `print(re.findall(" is",String1+String2))`

    _____

34. `print(re.findall("((?:20|19)(?:\d{2}))",String1))`

    _____

35. `print(re.findall("(?:(?:in\s)([a-z]+))+",String2))`

    _____

```
=====================================================
                        (1)        (2)        (3)
-----------------------------------------------------
Intercept             46.3174*** 44.3875*** 42.9846***
                      (0.8231)   (1.0334)   (1.1249)
weight                -0.0077*** -0.0072*** -0.0068***
                      (0.0002)   (0.0003)   (0.0003)
weight:origin[T.EU]              0.0004     -0.0004
                                 (0.0003)   (0.0013)
weight:origin[T.Asia]            0.0008***  -0.0039**
                                 (0.0003)   (0.0016)
origin[T.EU]                                2.3912
                                            (3.4509)
origin[T.Asia]                              11.2755***
                                            (3.7058)
R-squared             0.6918     0.6988     0.7063
R-squared Adj.        0.6910     0.6965     0.7026
=====================================================
Standard errors in parentheses.
* p<.1, ** p<.05, ***p<.01
```

E. [15 Points] Fix the code below to get the table above.

```python
import pandas as pd
from statsmodels.iolib.summary2 import summary_col
auto = pd.read_csv("https://www.statlearning.com/s/Auto.csv")

codes = ["Domestic", "EU", "Asia"]

auto["origin"] = pd.Categorical.from_codes(auto["origin"]-1, codes)

model1 = smf.ols("mpg ~ weight", data=auto).fit(cov_type="HC1")
model2 = smf.ols("mpg ~ weight + weight*origin", data=auto).fit(cov_type="HC1")
model3 = smf.ols("mpg ~ origin weight:origin", data=auto).fit(cov_type="HC1")

print(summary_col([model1, model2, model3], stars=True, float_format="%0.2f",
                  regressor_order=["origin", "weight", "Intercept"],
                  model_names=["(1)","(2)", "(3)"]))
```